You are a senior full-stack engineer, children's educational product designer, phonics specialist, and UX designer.

Build a polished web application that I can use with my five-year-old son, who turns six in October, to help him learn to read.

The working title is **WordQuest**, but structure the project so the name and branding can easily be changed.

1. Product vision

Create a warm, playful, visually impressive reading app designed for a parent and child to use together.

The app should help my son:

- Recognise letters and their sounds.
- Blend sounds into words.
- Learn high-frequency sight words.
- Read short sentences.
- Read simple illustrated stories.
- Read words and sentences aloud.
- Build confidence through encouragement and visual rewards.
- Review words he finds difficult.
- Progress through structured lessons rather than random activities.

The app should feel like a small adventure game, not like school homework.

It must be simple enough for a five-year-old to navigate, while giving the parent visibility and control over lessons and progress.

2. Core experience

The child enters a colourful world containing a map with different learning areas.

Possible areas include:

1. Letter Lagoon
2. Sound Forest
3. Word Village
4. Sentence Mountain
5. Story Castle

Each area contains lesson nodes. Completing a lesson unlocks the next lesson.

A typical session should last between 10 and 15 minutes.

Each session should contain:

1. A cheerful welcome.
2. A quick review of previously learned material.
3. Introduction of one new reading concept.
4. Guided practice.
5. A read-aloud activity.
6. A small game or challenge.
7. A reward screen.
8. A clear stopping point.

Do not overwhelm the child with too many controls, animations, words, or choices.

3. Learning philosophy

Use a systematic, progressive reading approach based primarily on phonics.

The learning sequence should generally be:

1. Letter recognition.
2. Letter sounds.
3. Sound discrimination.
4. Oral blending.
5. CVC words.
6. Digraphs.
7. Common consonant blends.
8. Sight words.
9. Short sentences.
10. Decodable stories.
11. Reading comprehension.

Teach lowercase letters before placing too much emphasis on uppercase letters, since lowercase letters appear more frequently in normal reading.

Introduce sounds in an order that allows useful words to be formed early.

Use an initial sound order similar to:

- s
- a
- t
- p
- i
- n
- m
- d
- g

- o
- c
- k
- e
- u
- r
- h
- b
- f
- l
- j
- v
- w
- x
- y
- z
- q

The exact order should be configurable through lesson data rather than hardcoded into components.

4. Lesson structure

Every lesson must be represented as structured data.

Create a reusable lesson engine instead of hardcoding every lesson screen.

A lesson should support steps such as:

- introduction
- listen
- repeat
- sound selection
- letter selection
- picture matching
- word building
- sound blending
- sight-word practice
- read aloud
- sentence reading
- story page
- comprehension question
- celebration

Suggested lesson data shape:

```
type Lesson = {
  id: string;
```

```
  title: string;
  description: string;
  worldId: string;
  order: number;
  difficulty: number;
  estimatedMinutes: number;
  prerequisites: string[];
  objectives: string[];
  newLetters: string[];
  newSounds: string[];
  newWords: string[];
  reviewWords: string[];
  steps: LessonStep[];
  reward: RewardDefinition;
};
```

Create discriminated TypeScript types for each lesson-step variant.

Each step must be rendered by the lesson engine using the appropriate interactive component.

5. Initial curriculum

Create enough real lesson data to demonstrate a coherent learning path.

Do not populate the app with meaningless placeholder lessons.

World 1: Letter Lagoon

Teach visual letter recognition and sounds.

Example lessons:

- Meet the sound “s”
- Meet the sound “a”
- Meet the sound “t”
- Find the matching letter
- Which picture starts with this sound?
- Review: s, a and t
- Meet the sound “p”
- Meet the sound “i”
- Meet the sound “n”
- Review: p, i and n

World 2: Sound Forest

Teach sound blending and segmentation.

Example lessons:

- Blend “s-a-t”
- Read “sat”
- Read “sit”
- Read “pin”
- Read “tap”
- Read “nap”
- Listen for the first sound
- Listen for the final sound
- Build a word from sound tiles

World 3: Word Village

Focus on CVC words and early sight words.

Include word families such as:

- -at: cat, mat, sat, pat
- -in: pin, tin, fin
- -an: man, fan, pan
- -it: sit, fit, hit
- -op: top, hop, mop
- -og: dog, log, fog

Introduce common sight words gradually:

- I
- a
- the
- is
- my
- to
- go
- we
- see
- can
- like
- and

Do not introduce too many sight words in one lesson.

World 4: Sentence Mountain

Teach short decodable sentences.

Examples:

- A cat sat.
- The dog can run.
- I can hop.
- We see a red bus.
- My hat is big.
- The pig is in the mud.

Use illustrations to support meaning, but do not make it possible to guess every word solely from the picture.

World 5: Story Castle

Provide very short decodable stories.

Each story should:

- Use mostly previously learned sounds and words.
- Contain between four and eight short pages.
- Include one sentence or a few short sentences per page.
- Include illustrations.
- End with one or two simple comprehension questions.
- Allow the parent to mark whether the child read independently, needed help, or listened.

Create at least two sample stories.

6. Lesson activity types

Implement reusable versions of the following activities.

Listen and repeat

- Show a large letter or word.
- Play its pronunciation.
- Invite the child to say it.
- Provide a large microphone button.
- Where browser speech recognition is supported, attempt to recognise the response.
- Where it is unsupported or unreliable, allow the parent to confirm:
- Correct
- Almost
- Try again

Never punish the child for speech-recognition errors.

Tap the correct sound

- Play a sound.

- Display three or four large choices.
- Let the child select the matching letter.
- Give immediate visual and audio feedback.

Picture-to-sound matching

- Show simple illustrated objects.
- Ask which object begins with a target sound.
- Use age-appropriate, unambiguous vocabulary.

Build the word

- Display draggable or tappable letter tiles.
- Let the child arrange the letters into the correct word.
- Include a button that slowly plays each phoneme.
- Animate the tiles joining together when the sounds are blended.

Read the word

- Display a large word.
- Initially show sound buttons or phoneme segmentation.
- Let the child tap each sound.
- Then animate the sounds blending into the complete word.
- Finally invite the child to read the word aloud.

Sentence reading

- Display one clear sentence.
- Highlight each word as the app reads it.
- Let the child tap an individual word for help.
- Tapping a word must not automatically mark the sentence as complete.
- Allow the parent to record whether the child read independently or needed support.

Story reading

- Use large illustrated story cards.
- Include forward and back navigation.
- Provide optional narration.
- Track which pages were read.
- End with a celebration and comprehension check.

7. Read-aloud functionality

Use browser capabilities carefully.

Implement:

- Speech synthesis through the Web Speech API.

- Configurable voice, speaking rate, and volume.
- Slow pronunciation mode for individual sounds and words.
- Normal narration mode for sentences and stories.
- Speech recognition where supported.
- Graceful fallback when speech recognition is unavailable.
- Clear permission handling for microphone access.
- No automatic recording.
- No audio uploads.
- No permanent storage of the child’s voice in the MVP.

Create a speech-service abstraction so browser speech can later be replaced by a higher-quality speech API.

For phonemes that normal text-to-speech pronounces poorly, allow lesson data to reference pre-recorded audio files.

8. Rewards and motivation

Create a visual reward system that encourages consistency without creating pressure.

Possible rewards include:

- Stars
- Gems
- Stickers
- Character accessories
- World decorations
- Badges
- Storybook pieces
- A growing magical tree

The child should earn rewards for:

- Completing a lesson.
- Trying again after a mistake.
- Reading aloud.
- Completing a review.
- Finishing a story.
- Maintaining a gentle learning streak.

Do not subtract rewards for incorrect answers.

Do not use aggressive streak-loss messaging.

Examples of encouraging messages:

- “Great trying!”
- “You worked that out!”

- “Let’s sound it out together.”
- “That was a tricky word.”
- “You remembered it!”
- “Amazing reading!”

Create a reward-room screen where the child can see earned stickers, badges, and decorations.

Include at least:

- First Lesson badge
- Five Words badge
- Brave Reader badge
- Story Explorer badge
- Sound Master badge
- Three-Day Adventure badge

9. Adaptive review

Track performance at the letter, sound, word, sentence, and lesson level.

For each learning item, store:

- Number of attempts.
- Number of successful attempts.
- Last practised date.
- Whether help was needed.
- Current familiarity level.
- Recent mistakes.
- Whether the item should appear in review.

Use a simple spaced-review model.

Suggested familiarity states:

- new
- learning
- practising
- familiar
- mastered

Items answered incorrectly or completed with help should reappear sooner.

Mastered items should still appear occasionally.

At the beginning of each session, create a short review containing three to five items based on:

1. Recent difficulty.

2. Time since last practice.
3. Relevance to the upcoming lesson.
4. Previously mastered items due for maintenance review.

Keep the first implementation deterministic and understandable. Do not add machine learning.

10. Parent mode

Include a parent area protected by a simple parent gate, such as holding a button for three seconds or solving a basic adult-oriented prompt.

The parent dashboard should show:

- Current lesson.
- Completed lessons.
- Letters learned.
- Words learned.
- Words needing review.
- Reading streak.
- Total reading time.
- Recent activity.
- Accuracy trends.
- Stories completed.
- Badges earned.

Allow the parent to:

- Choose the next lesson.
- Repeat a lesson.
- Mark a lesson complete.
- Reset lesson progress.
- Add a custom word.
- Create a small custom word list.
- Select lesson duration.
- Enable or disable speech recognition.
- Change narration speed.
- Turn sound effects on or off.
- Choose whether lessons unlock sequentially.
- Export progress as JSON.
- Import previously exported progress.

Do not present the parent with overly complex analytics.

11. Child profile

For the MVP, support one local child profile.

Profile fields should include:

- Display name
- Avatar
- Birth month and year
- Preferred session duration
- Current curriculum position
- Rewards
- Learning progress
- Settings

Do not require the child’s full legal name.

Do not require an online account for the MVP.

Store all progress locally in IndexedDB, with a clean persistence abstraction that can later be connected to a backend.

12. Visual design

The visual quality is important.

The app should feel modern, warm, magical, and premium.

Design principles:

- Large touch targets.
- Minimal text on navigation screens.
- Rounded cards.
- Expressive character animation.
- Smooth transitions.
- Clear hierarchy.
- High contrast.
- Child-friendly typography.
- No clutter.
- No advertisements.
- No dark patterns.
- No external links in child mode.
- No competitive leaderboards.
- No countdown timers that create anxiety.

Use animations purposefully:

- Stars flying into a reward counter.
- Letter tiles bouncing gently.
- Correct answers creating a sparkle effect.
- A world node lighting up when unlocked.

- A character celebrating lesson completion.
- A magical tree growing as lessons are completed.

Respect `prefers-reduced-motion`.

Avoid excessive animation during reading activities.

13. Mascot

Create an original friendly mascot, such as a small explorer fox, lion cub, or robot.

The mascot should:

- Welcome the child.
- Explain activities using very short sentences.
- Celebrate effort.
- Offer hints.
- Avoid negative reactions.
- Appear consistently throughout the app.

Use an original SVG-based mascot or simple CSS illustration so the app works without copyrighted assets.

Do not use copyrighted characters.

14. Accessibility

The app must be accessible to children and adults.

Include:

- Keyboard support.
- Screen-reader labels.
- Visible focus states.
- Sufficient colour contrast.
- Captions or visible text for important audio.
- Controls for sound and narration.
- Reduced-motion support.
- Buttons that are easy to understand without reading.
- No interaction based solely on colour.
- Proper semantic HTML.
- Appropriate ARIA only where necessary.

Test major flows using Playwright and axe-core.

15. Technical architecture

Use:

- Next.js with the App Router.
- TypeScript with strict mode.
- React.
- Tailwind CSS.
- Framer Motion for restrained animation.
- Zustand or a similarly lightweight state-management solution.
- IndexedDB using Dexie.
- Zod for runtime validation of lesson and progress data.
- React Hook Form for parent forms.
- Vitest and React Testing Library.
- Playwright for end-to-end tests.
- axe-core for automated accessibility checks.
- ESLint and Prettier.

The app must run with:

```
npm install
npm run dev
```

Do not introduce a backend, authentication service, payment system, analytics platform, or cloud dependency in the initial implementation.

Use a clean repository structure such as:

```
src/
  app/
  components/
    child/
    parent/
    lessons/
    rewards/
    speech/
    shared/
  curriculum/
    worlds/
    lessons/
    stories/
  domain/
    lesson/
    progress/
    rewards/
```

```
    review/
  services/
    persistence/
    speech/
    audio/
  stores/
  hooks/
  lib/
  styles/
  test/
```

Keep domain logic separate from visual components.

16. Main routes

Create these routes:

```
/
 /profile
 /adventure
 /lesson/[lessonId]
 /review
 /rewards
 /stories
 /stories/[storyId]
 /parent
 /parent/progress
 /parent/curriculum
 /parent/settings
```

The root route should continue the child's current adventure rather than looking like a corporate landing page.

17. Home and adventure screens

The child's home screen should show:

- Mascot greeting.
- Current world.
- Large "Continue Adventure" button.
- Current star or gem total.
- One small review prompt.
- Recently earned badge.
- Parent-mode access placed discreetly.

The adventure map should show:

- Worlds.
- Completed lesson nodes.
- Current lesson node.
- Locked future lessons.
- Story checkpoints.
- Reward chests.

The map may initially be a vertical or winding two-dimensional path. Do not build a complex game engine.

18. Progress model

Create strongly typed progress models.

Track:

```
type AttemptResult = {
  itemId: string;
  itemType: "letter" | "sound" | "word" | "sentence" | "story";
  lessonId: string;
  timestamp: string;
  correct: boolean;
  helpLevel: "none" | "hint" | "parent";
  responseSource: "tap" | "speech" | "parent-confirmation";
  durationMs?: number;
};
```

Lesson progress should include:

- not_started
- in_progress
- completed
- repeated

A lesson must save progress after every step so an interrupted session can resume safely.

19. Safety and privacy

This is a child-focused application.

Apply these rules:

- Local-first storage.
- No advertisements.
- No social features.

- No public profiles.
- No messaging.
- No behavioural tracking.
- No unnecessary personal data.
- No third-party analytics.
- No automatic microphone recording.
- Ask for microphone permission only when the user starts a read-aloud activity.
- Explain microphone use clearly to the parent.
- Keep parent controls separate from child controls.

20. Seed content

Include:

- At least 15 complete lessons.
- At least 30 practice words.
- At least 12 sight words.
- At least 15 example sentences.
- At least two complete decodable stories.
- At least six badges.
- At least 12 collectible stickers or decorations.
- At least three review activity formats.

The seed content should be educationally coherent and usable, not filler.

Use simple words and illustrations appropriate for a child in Zimbabwe, while keeping the curriculum internationally understandable.

Objects may include familiar examples such as:

- bus
- sun
- cup
- cat
- dog
- hat
- mat
- pan
- pot
- fish
- tree
- car
- book
- drum
- mango

Avoid vocabulary that depends heavily on unfamiliar cultural references.

21. Testing requirements

Write tests for:

- Lesson schema validation.
- Lesson prerequisite checking.
- Lesson unlocking.
- Progress persistence.
- Review-item selection.
- Familiarity-state transitions.
- Reward granting.
- Prevention of duplicate rewards.
- Lesson resume behaviour.
- Speech-recognition fallback.
- Parent-setting changes.
- Import and export validation.

Create Playwright tests for:

1. Creating the child profile.
2. Starting the first lesson.
3. Completing a lesson.
4. Receiving a reward.
5. Resuming an interrupted lesson.
6. Completing a review.
7. Reading a story.
8. Opening parent mode.
9. Viewing difficult words.
10. Changing narration settings.

22. Implementation process

Work autonomously and make sensible product decisions.

Proceed in this order:

1. Inspect the existing repository.
2. Create or update the implementation plan.
3. Establish the design system.
4. Define domain models and schemas.
5. Implement persistence.
6. Create the lesson engine.
7. Add initial curriculum data.
8. Build the child experience.
9. Build the reward system.
10. Add speech features and fallbacks.
11. Build parent mode.

12. Add adaptive review.
13. Add tests.
14. Run linting, type checking, tests, and production build.
15. Fix all significant failures.
16. Document the project.

Do not stop after generating a plan or mock-up. Implement a complete working MVP.

23. Engineering standards

- Avoid giant components.
- Avoid `any`.
- Avoid duplicated lesson logic.
- Avoid hardcoded progress rules inside UI components.
- Avoid storing derived state unnecessarily.
- Validate persisted data.
- Handle corrupted or outdated local data gracefully.
- Add a persistence schema version and migration mechanism.
- Make lesson content easy to edit without changing UI code.
- Ensure components work on mobile, tablet, and desktop.
- Optimise primarily for tablet use by a parent and child sitting together.
- Provide loading, empty, error, and recovery states.
- Use original local SVG illustrations or generated abstract illustrations.
- Do not depend on remote image URLs for core functionality.

24. Documentation

Create a detailed README containing:

- Product overview.
- Setup instructions.
- Available scripts.
- Architecture.
- Curriculum structure.
- How to add a lesson.
- How to add an activity type.
- How to add a story.
- How progress is calculated.
- How adaptive review works.
- Speech-browser limitations.
- Accessibility notes.
- Data-storage and privacy notes.
- Future backend integration points.

Also create:

```
docs/
  PRODUCT_SPEC.md
  CURRICULUM_GUIDE.md
  LESSON_AUTHORING_GUIDE.md
  ARCHITECTURE.md
```

25. Definition of done

The MVP is complete when:

- A child profile can be created.
- The child can see an adventure map.
- Lessons unlock progressively.
- At least 15 lessons are fully playable.
- Lessons include sound, word, sentence, and read-aloud activities.
- Progress survives browser refreshes.
- Interrupted lessons can resume.
- Difficult words appear in future reviews.
- Rewards and badges are visibly earned.
- At least two stories are readable.
- Parent mode displays meaningful progress.
- The app works without a backend.
- The app is responsive.
- Core functionality works without speech recognition.
- Automated tests pass.
- Type checking passes.
- The production build succeeds.
- The README explains how to run and extend the application.

Begin by inspecting the repository and creating a concise implementation plan. Then build the complete MVP without waiting for additional approval.